

JCoder-1 让Agent开口说话

背景介绍：由于我们会使用到虚拟线程,建议 **Java 21** 以上的版本,

教学使用 **java 21**

环境配置

使用到两个库：

```
XML
<dependencies>
  <!-- JSON + ObjectMapper -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.18.3</version>
  </dependency>

  <dependency>    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.4</version>
    <scope>test</scope>
  </dependency>

</dependencies>
```

自己new 一个Maven工程,那么接下来我们开始实现

实现访问LLM

我们这一章节做的事情就只有几件：让LLM开口说话

我们可以想一想,我们经常使用的 Claude等其他产品

我们要做的就是

- 实现一个http访问+SSE流式输出
- 实现配置读取api-key
- 实现简单的记忆保存

知识讲解：常见请求格式

大模型应用开发必读：[OpenAI 接口格式全方位详解与生产最佳实践-CSDN博客](#)

我觉得看完这些文章之后应该会更通透

请求头

YAML

```
Base-URL: https://api.openai.com/v1
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json    # 流式输出：必须
Accept: text/event-stream        # 流式输出：非必须
```

常见接口总览

接口	请求方式	路径	主要用途
模型列表	GET	/v1/models	查询可用模型
Chat Completions	POST	/v1/chat/completions	多轮对话、文本生成、工具调用
Responses	POST	/v1/responses	新一代统一生成接口，支持文本、图像、工具等多模态输入输出
Embeddings	POST	/v1/embeddings	文本向量化，用于检索、聚类 and 相似度计算

接口	请求方式	路径	主要用途
Images	POST	/v1/images/generations	图像生成，具体字段因模型而异
Audio	POST	/v1/audio/transcriptions	语音转文本
Files	POST	/v1/files	上传文件，用于微调、批处理、助手等场景
Batch	POST	/v1/batches	创建批量异步任务

普通调用

示例演示：见 `post-man`

// 最小请求演示

```
{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "user",
      "content": "用一句话解释什么是向量数据库"
    }
  ]
}
```

// response

```
{
  "id":
  "resp_0f450cc6de1d01b0016a6460885f2881989640c34a92c08140",
  "object": "chat.completion",
  "created": 1784963208, // Unix 时间戳
  "model": "gpt-5.6-sol",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "向量数据库是一种专门存储、检索和管理向量数据的数据库，能够通过计算向量之间的相似度快速找到语义相近的信息。",
        "reasoning_content": null,
        "tool_calls": null
      },
      "finish_reason": "stop", // 见下(important)
      "native_finish_reason": "stop"
    }
  ],
  "usage": {
    "completion_tokens": 42,
```

```

    "total_tokens": 57,
    "prompt_tokens": 15,
    "prompt_tokens_details": {
        "cached_tokens": 0
    },
    "completion_tokens_details": {
        "reasoning_tokens": 0
    }
}
}

```

chat/completions 参数

参数	类型	默认值	说明
<code>model</code>	<code>string</code>	无	模型 ID，必填
<code>messages</code>	<code>array</code>	无	对话上下文，必填
<code>temperature</code>	<code>number</code>	通常为 <code>1</code>	控制随机性，数值越高越发散
<code>top_p</code>	<code>number</code>	通常为 <code>1</code>	Nucleus sampling，通常不和 <code>temperature</code> 同时大幅调整
<code>max_tokens</code>	<code>integer</code>	模型默认	限制输出 Token 数，部分新接口使用 <code>max_output_tokens</code>
<code>stream</code>	<code>boolean</code>	<code>false</code>	是否使用流式输出
<code>stop</code>	<code>string/array</code>	<code>null</code>	遇到指定文本时停止输出
<code>presence_penalty</code>	<code>number</code>	<code>0</code>	鼓励引入新主题
<code>frequency_penalty</code>	<code>number</code>	<code>0</code>	减少重复表达
<code>tools</code>	<code>array</code>	<code>null</code>	可调用工具定义
<code>tool_choice</code>	<code>string/object</code>	<code>auto</code>	控制是否调用工具
<code>response_format</code>	<code>object</code>	<code>null</code>	控制结构化输出格式

参数	类型	默认值	说明
<code>seed</code>	<code>integer</code>	<code>null</code>	尝试获得更稳定的采样结果，非绝对确定

核心响应字段

字段	类型	说明
<code>id</code>	<code>string</code>	本次请求的唯一 ID
<code>object</code>	<code>string</code>	对象类型，如 <code>chat.completion</code>
<code>created</code>	<code>integer</code>	Unix 时间戳
<code>model</code>	<code>string</code>	实际使用的模型
<code>choices</code>	<code>array</code>	候选输出列表
<code>choices[].message</code>	<code>object</code>	模型回复消息
<code>choices[].finish_reason</code>	<code>string</code>	结束原因
<code>usage</code>	<code>object</code>	Token 用量统计

`finish_reason` 常见值

这个字段用来判断 SSE 是否结束,如果是 `null` 就没结束, `stop` 就代表结束了

值	含义	处理建议
<code>stop</code>	正常停止	可直接使用结果
<code>length</code>	达到 Token 限制	增大 Token 限制或继续请求
<code>tool_calls</code>	模型请求调用工具	执行工具后，把结果放回 <code>messages</code>
<code>content_filter</code>	内容被安全策略拦截	调整输入或提示用户
<code>null</code>	流式输出尚未结束	继续读取流

流式调用

// 发起请求

```
{
  "model": "gpt-5.6-sol",
  "messages": [
    {"role": "user", "content": "写一段 200 字的产品介绍"}
  ],
  "stream": true
}
```

//结果

```
data: {
  "id":
"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb02a1",
  "object": "chat.completion.chunk",
  "created": 1784964383,
  "model": "gpt-5.6-sol",
  "choices": [
    {
      "index": 0,
      "delta": {
        "role": "assistant",
        "reasoning_content": "***Planning 200-character
Chinese product intro**"
      },
      "finish_reason": null,
      "native_finish_reason": null
    }
  ]
}
```

```
data:
{"id":"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb02a1","object":"chat.completion.chunk","created":1784964383,"mod
```

```
el":"gpt-5.6-sol","choices":[{"index":0,"delta":
{"role":"assistant","reasoning_content":"\n\n"},"finish_reason":null,"n
ative_finish_reason":null}]}
```

```
data:
{"id":"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb
02a1","object":"chat.completion.chunk","created":1784964383,"mod
el":"gpt-5.6-sol","choices":[{"index":0,"delta":
{"role":"assistant","content":"这"},"finish_reason":null,"native_finish_r
eason":null}]}
```

```
data:
{"id":"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb
02a1","object":"chat.completion.chunk","created":1784964383,"mod
el":"gpt-5.6-sol","choices":[{"index":0,"delta":
{"role":"assistant","content":"款"},"finish_reason":null,"native_finish_r
eason":null}]}
```

```
// 省略,中间太多了
```

```
//
```

```
//
```

```
//
```

```
//
```

```
//
```

```
//
```

```
data:
{"id":"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb
02a1","object":"chat.completion.chunk","created":1784964383,"mod
el":"gpt-5.6-sol","choices":[{"index":0,"delta":
{"role":"assistant","content":"选
择"},"finish_reason":null,"native_finish_reason":null}]}
```

```
data:
{"id":"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb
02a1","object":"chat.completion.chunk","created":1784964383,"mod
el":"gpt-5.6-sol","choices":[{"index":0,"delta":
{"role":"assistant","content":"。"},"finish_reason":null,"native_finish_r
```

```
eason":null}}}  
  
data: {  
  "id":  
"resp_0f9b9e9022a2e36e016a64651f642c819981a0e76916bb02a1"  
  ,  
  "object": "chat.completion.chunk",  
  "created": 1784964383,  
  "model": "gpt-5.6-sol",  
  "choices": [  
    {  
      "index": 0,  
      "delta": {},  
      "finish_reason": "stop",  
      "native_finish_reason": "stop"  
    }  
  ],  
  "usage": {  
    "completion_tokens": 281,  
    "total_tokens": 296,  
    "prompt_tokens": 15,  
    "prompt_tokens_details": {  
      "cached_tokens": 0  
    },  
    "completion_tokens_details": {  
      "reasoning_tokens": 88  
    }  
  }  
}
```

提取后的内容拼接：

这款智能降噪无线耳机，采用人体工学设计，轻盈小巧，佩戴舒适，适合通勤、运动、旅行及日常办公使用。搭载先进主动降噪技术，可有效降低地铁、街道和办公室等环境噪音，让音乐与通话更加清晰。高品质音频单元带来饱满低音、细腻人声和宽广声场，支持智能触控、双设备连接及高清通话功能。单次充电可持续使用长达八小时，搭配便携充电盒，续航时间更长。耳机支持快速配对，并具备防汗防水性能，满足多种使用场景。简约时尚的外观设计，兼顾科技感与实用性，是品质生活与高效工作的理想选择。

用ds试试

我们在这里再换为Deepseek自己去试一试

chat

这里ds多了一

个"`system_fingerprint`": "`fp_9954b31ca7_prod0820_fp8_kvcache_20260402`"

查看官网解释：表示模型运行时所使用的后端配置指纹

Responses

200 (No streaming)

200 (Streaming)

OK, 返回一个 `chat completion` 对象。

APPLICATION/JSON

Schema Example (from schema) Example

▼ SCHEMA

id string	REQUIRED
该对话的唯一标识符。	
> choices object[]	REQUIRED
created integer	REQUIRED
创建聊天完成时的 Unix 时间戳 (以秒为单位) 。	
model string	REQUIRED
生成该 completion 的模型名。	
system_fingerprint string	REQUIRED
This fingerprint represents the backend configuration that the model runs with.	
object string	REQUIRED
Possible values: [chat.completion]	
对象的类型, 其值为 chat.completion。	
> usage object	

SSE

```
data: {
  "id": "660d5e93-73e2-45cc-9d6c-a8f1d855cd33",
  "object": "chat.completion.chunk",
  "created": 1784965167,
  "model": "deepseek-v4-pro",
  "system_fingerprint":
"fp_9954b31ca7_prod0820_fp8_kvcache_20260402",
  "choices": [
    {
      "index": 0,
      "delta": {
        "content": "",
        "reasoning_content": null
      },
      "logprobs": null,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 3625,
    "total_tokens": 3637,
    "prompt_tokens_details": {
      "cached_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 3465
    },
    "prompt_cache_hit_tokens": 0,
    "prompt_cache_miss_tokens": 12
  }
}
```

data: [DONE]

ds会在结束后返回一个 `data: [DONE]`

这个便于我们知道SSE结束了,但是gpt确实没有.

gpt主要通过 `finish_reason` 字段判断,ds有双重判断

工具调用是怎么实现的?

- 我们这章主要用 `chat` 演示,而不用 `SSE`,主要是好看
- 我会优先使用 `OpenAI-api`,而不使用 `ds`,大家使用ds的时候请锻炼一下自己的问题解决能力(其实是因为plus会员额度还没用完,不用完感觉就是吃亏)

谜底出在谜面上,再次查看 `chat/completions` 的参数

我们现在来学习 `tools`

比如,当我们说"请查看当前文件夹的文件,并帮我修改"
模型是如何知道要使用工具,使用什么工具呢?

其实用一句话来说,就是我们给模型传入一个 `tools`,模型会自己判断是否使用工具,之后返回文字,然后我们本地Agent解析出来 `tool_use`,在本地调用对应工具,得到结果,再把结果返回给Agent.

这样,LLM就"学会了"如何使用工具

这个过程中,我们相当于是"执行者",LLM是决策者

```
// 发出请求
{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "user",
      "content": "北京今天适合跑步吗？"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "查询指定城市的天气",
        "parameters": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "城市名称"
            }
          },
          "required": ["city"]
        }
      }
    }
  ],
  "tool_choice": "auto"
}
```

// 返回结果

```
{
  "id":
    "resp_0c76004b92501c74016a64710c7ba4819ab4f09b90f84e8faf"
  ,
  "object": "chat.completion",
  "created": 1784967436,
  "model": "gpt-5.6-sol",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "reasoning_content": "**Planning weather tool
integration**",
        "tool_calls": [
          {
            "id": "call_dsXqkZwWnVOUKZu3po9JuLqo",
            "type": "function",
            "function": {
              "name": "get_weather",
              "arguments": "{\"city\":\"北京\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls",
      "native_finish_reason": "tool_calls"
    }
  ],
  "usage": {
    "completion_tokens": 30,
    "total_tokens": 85,
    "prompt_tokens": 55,
    "prompt_tokens_details": {
      "cached_tokens": 0
    }
  }
}
```

```
    },  
    "completion_tokens_details": {  
      "reasoning_tokens": 10  
    }  
  }  
}
```

我们可以看到:在 **tool_calls** 字段有我们想要的结果
我们把自己执行后的结果包装进去

```
// 封装messages后再次发出请求
{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "user",
      "content": "北京今天适合跑步吗？"
    },
    {
      "role": "assistant",
      "content": null,
      "reasoning_content": "***Planning weather tool integration**",
      "tool_calls": [
        {
          "id": "call_dsXqkZwWnVOUKZu3po9JuLqo",
          "type": "function",
          "function": {
            "name": "get_weather",
            "arguments": "{\"city\":\"北京\"}"
          }
        }
      ]
    },
    {
      "role": "tool",
      "tool_call_id": "call_dsXqkZwWnVOUKZu3po9JuLqo",
      "content": "{\"city\":\"北京\", \"temperature\":28, \"air_quality\":\"良\", \"suggestion\":\"适合傍晚慢跑\"}"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_weather",
        "description": "查询指定城市的天气",

```

```
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string",
          "description": "城市名称"
        }
      },
      "required": ["city"]
    }
  },
  "tool_choice": "auto"
}
```

```
// 返回结果
{
  "id":
    "resp_08bbe7fe1c3be3c9016a6474af828c819b8cd34f545c046242"
  ,
  "object": "chat.completion",
  "created": 1784968367,
  "model": "gpt-5.6-sol",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "北京今天适合跑步。当前约 **28°C**，空气质  
量 **良**，建议选择**傍晚**气温较低时慢跑；注意补水，避免正午高  
温时段。",
        "reasoning_content": null,
        "tool_calls": null
      },
      "finish_reason": "stop",
      "native_finish_reason": "stop"
    }
  ],
  "usage": {
    "completion_tokens": 51,
    "total_tokens": 160,
    "prompt_tokens": 109,
    "prompt_tokens_details": {
      "cached_tokens": 0
    },
    "completion_tokens_details": {
      "reasoning_tokens": 0
    }
  }
}
```

我们上面写成了一个Function的形式,也有其他不同的格式,例如

```
// tools 描述
{
  "tools": [{
    "name": "ReadFile",
    "description": "读取指定路径的文件内容。返回带行号的文件文本。
    路径必须是绝对路径。",
    "input_schema": {
      "type": "object",
      "properties": {
        "path": {
          "type": "string",
          "description": "文件的绝对路径"
        }
      },
      "required": ["path"]
    }
  ]
}
```

```
// tool_result封装 request
{
  "role": "user",
  "content": [{
    "type": "tool_result",
    "tool_use_id": "tool_123",
    "content": "1\tdef main():\n2\t    print('hello')\n3\t"
  ]
}
```

工具声明的不同格式

标准API风格(OpenAI Function Calling)

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "查询指定城市的实时天气信息，包括温度、湿度和天气状况。适用于用户询问某地天气时调用。",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string",
          "description": "城市名称，如'北京'、'上海'。支持国内主要城市。",
          "enum": ["北京", "上海", "广州", "深圳"] // 可选项
        },
        "unit": {
          "type": "string",
          "description": "温度单位，可选摄氏度（metric）或华氏度（imperial），默认为metric。",
          "default": "metric"
        }
      },
      "required": ["city"]
    },
    "returns": {
      "type": "object",
      "properties": {
        "temperature": {"type": "number"},
        "humidity": {"type": "number"},
        "condition": {"type": "string"}
      }
    }
  }
}
```

```
}  
}
```

简化工具格式

```
{  
  "tools": [{  
    "name": "ReadFile",  
    "description": "读取指定路径的文件内容，返回带行号的文件文本。适用于需要分析代码、日志或配置文件时。文件路径必须是绝对路径。",  
    "input_schema": {  
      "type": "object",  
      "properties": {  
        "path": {  
          "type": "string",  
          "description": "文件的绝对路径，例如 /home/user/data.txt。不支持相对路径。"  
        }  
      },  
      "required": ["path"]  
    }  
  }  
}]  
}
```

所以很简单可以看出,工具调用实现十分简单,我们是执行者,LLM只是决策者

其他扩展

结构化输出JSON

如果希望Agent返回的结果是固定的json的话,我们可以用下面的做法,依旧利用 `chat` 的参数

`reponse_format`

```

{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "user",
      "content": "从句子中抽取姓名和城市：张三住在杭州。"
    }
  ],
  "response_format": {
    "type": "json_object"
  }
}

```

部分模型支持更严格的 **JSON Schema**

```

{
  "response_format": {
    "type": "json_schema",
    "json_schema": {
      "name": "person_info",
      "schema": {
        "type": "object",
        "properties": {
          "name": {"type": "string"},
          "city": {"type": "string"}
        },
        "required": ["name", "city"],
        "additionalProperties": false
      }
    }
  }
}

```

懒得测了

Responses

这个是新的接口,支持多模态.

是把文本生成、多模态输入、工具调用等能力统一到一个更灵活的格式中

```
// 给个示例
{
  "model": "gpt-5.6-sol",
  "input": [
    {
      "role": "user",
      "content": [
        {
          "type": "input_text",
          "text": "请分析这张图中的主要信息,并告诉我是哪个动漫"
        },
        {
          "type": "input_image",
          "image_url": "https://ts1.tc.mm.bing.net/th/id/R-
C.0c6fbbcb328d0c9e1d2d17893b27660b?
rik=nvl%2fHBYgoeElvA&riu=http%3a%2f%2fn.sinaimg.cn%2fsinakd2
0240906ac%2f205%2fw1080h725%2f20240906%2f66c1-
652adf2d3f80ffdc8684a57887dc31d7.jpg&ehk=gu9dwo0uC5g1%2b
MiLzTMBQIn3NrQXLT7iE0rXVZOfk1M%3d&risl=&pid=ImgRaw&r=0"
        }
      ]
    }
  ]
}
```

```
{
  "id":
"resp_0100a392b848ed0c016a6479dc6f54819b97a3b4dc589fb809",
  "object": "response",
  "created_at": 1784969692,
  "status": "completed",
  "background": false,
  "completed_at": 1784969703,
  "error": null,
  "frequency_penalty": 0.0,
  "incomplete_details": null,
  "instructions": null,
  "max_output_tokens": null,
  "max_tool_calls": null,
  "model": "gpt-5.6-sol",
  "moderation": null,
  "output": [
    {
      "id":
"rs_0100a392b848ed0c016a6479e70120819bbd02aa6a6eb5d294",
      "type": "reasoning",
      "content": [],
      "encrypted_content":
"gAAAAABqZHnn_yE0uNJ0wktQSRqJlnOpPt6rsytlrboZwK-
wqqxMraAKuMgTDDNXuF9ZnT9paC_gq_x0aVpoPyXOT_Px-
BsbKcM0e_3BtkAxOtxZbpJJcz47TDvee8Mcf2SyrYN5aPyaGMZktge7
mbYL_dr57VSU6gQWVHP39h0WSILJSqE8UN4MVjaVUcFd79YXWAA
rWaALrs0JX-
ydyX9u217NWICv00s0wb9WFGls0VeClGMm4iBoBs28zIV6qRp3qd2J
ajPPJ7ikmD1fzu3-
Z2mHavXRfljMdcHfninq_2m_MbSCTQaakaqw3yp8d4sY5-Fd-
j0NLKgVxmc0DoQZlyd-
BYYSydMhMgxJx56OuxOzoPc4ONoURFBjkMb-
uRP7peoXha55O8rL7aIRCFHw5TmWhhAWiYbg7AzqtN5_s-
wEilnzKpYGtbJZwxuuk_CRBQ8jPC_p_dPjcU_FXrBDnKa90xgEzNi4Y7
ZbKkcBmaJz_IIDPDVnMgeasi-1DXfmwywFw1aaZH6n2hcK1QuKLJx-
```

0_UO9vbu6crOyhUsQ9B9znGAmVwWBS_rRyaSjZpMsr_5FJdAr-
mdPC8EGVK5yfnmkbuzYDNUlHVdNee7Vhv6fLzdkmWZThyT5kDqkO
g5tLSeUwWal7sdB7rjZV9Pe4zd1sQWbQJ4e9mToK_mcjX79d3cYbJQ
hMLs3O7t6zDNE5tLb1TMJw83dUYZUj0TVK5rkCMfVr8rgE6XfRzwWc
j57V_v9T821QJnFJrjlzOL_pl5YFrg96nTvvOWQ1MHnQCTLI_XFugzLA
ElkJIU4SVKN-
OGXArIOgDFkTa9docUQqPBogtO7MJfxFm1ip_Ud9Tc5RbsW23KBUE
uhBq6Z0OITm7obpeB_TmhYhyF_CHP_aSVIRfm0y7Y-
uGgmmJGuOOZUaeO8oQ4fEjW6RI6QEi9h2KIUoEgVR6CFlc6ekfZUm
6CkSOjJW_y0jJIBY6WYOTa0oLgt9KsiXhOVSr7Y-fW_eIYtZ6-
UDXhmxuaWA3bSlkjZYQJsiTgkjEcWMaxURbfUCSYf4H_VLMOORE5K
93fxkynWGnCWwlyljAKHiPW3-
ZotXrloKpDa7suawxyKe2IVef0CpzAFjRbNCAeqjWmqzqyp6s2Zs3pT
MsVw4myQ_HPjgmnyTqhpfpveBkJwZGw3yHEtf3sjdp7RIh_aNd8_FSL
EdVp2WrOlP0ces7Exl7HaE0cvNtyMS5QjMUNn0D1Cp4Mm9y8lSn4Q
HOMQ08pvywq5nSLVTMLBJpsfMdHq44lleKNNiCl8Bb-
9QsRiXhcDpN8qeRNFn36hUPBas0iyov_tDxOzar_Y4VqFsAsLvWGOP
ZidnTbuv35Vu02fICS8Myxth2TqP77xB9VjuYHJoc2tv4Fc5kfWGY2v9
MyDZof36Bdf1Kl4NuU1oh5DxO3tg4nlC8tlru76nqJyABBGvJXoRU7o
DtSaP3W5o8OHjTBg8uSxFSeAn2BBfTr-
oQiCKW7Yuf0ii8JYsS3A2lwhqBVNgiobJeHLUZXEGQJkaHbvD-z-jEP-
IU9eB4eqdyRnSH3Z6UxTUTDyc96lfVGI090dw8D0K2rn5mJdzu4QcR
MSS5aLOGxZuxz4ZFByrzVY25UPUItSq8gakw4ZxHIRmaIEDaiwm3V7
L0V8OpblSIFk-
f6s8THBSagQDFxwEEIXDuMfRuV0g_OF9hdIHY_XG_PaBqK9uT9X9At
Ur-
vii7thwknccRZfluhe2lGMSA7p56dCXBywTkFejRa_zdiMYfZN0DdWDD
SWAmmGKAKZhLnvz5OQ7WJsddqi65PjrnIVwbJBrDj0AlzAUyjC-
Li9AnL1U=",

```
    "summary": [],  
  },  
  {  
    "id":  
"msg_0100a392b848ed0c016a6479e71b2c819b8ae3ecf47f5d4e65"  
,  
    "type": "message",  
    "status": "completed",  
    "content": [
```

```
{
  "type": "output_text",
  "annotations": [],
  "logprobs": [],
  "text": "这张图是日本动漫 **《跃动青春》**（日文：**スキップとローファー**，英文：**Skip and Loafer**）的宣传插画。
\n\n### 图片主要信息\n- 画面中有四位女高中生，穿着校服，背景像是在学校的天台或户外平台。
\n- 整体色彩明亮，人物表情轻松活泼，体现了青春、友情和校园日常的氛围。
\n- 右上角的日文标题可以辨认为 **「スキップとローファー」**。
\n- 四位角色大致是：\n  - **岩仓美津未**：画面中央、棕色短发的女生
  - **志摩聪介**相关角色群中的女同学们，包括：
    \n    - **江头美嘉**：右上方红棕色短发
    \n    - **村重结月**：左侧金发
    \n    - **久留米诚**：最左侧戴眼镜、黑色长发
\n\n### 动漫简介\n《跃动青春》改编自高松美咲创作的同名漫画，讲述来自乡下的少女**岩仓美津未**到东京读高中后，与同学们相处、成长并逐渐建立友情的校园故事。作品风格温暖写实，重点描写青春期的的人际关系与成长。"
}
],
"phase": "final_answer",
"role": "assistant"
}
],
"parallel_tool_calls": false,
"presence_penalty": 0.0,
"previous_response_id": null,
"prompt_cache_key": "fb825f2d-710d-4dc5-8b1d-b106f2a927ea",
"prompt_cache_retention": "24h",
"reasoning": {
  "context": "all_turns",
  "effort": "medium",
  "mode": "standard",
  "summary": null
},
"safety_identifier": "user-Cgm47ApObJUjH8x5fStPNRpA",
"service_tier": "default",
"store": false,
"temperature": 1.0,
```

```
"text": {
  "format": {
    "type": "text"
  },
  "verbosity": "medium"
},
"tool_choice": "auto",
"tool_usage": {
  "image_gen": {
    "input_tokens": 0,
    "input_tokens_details": {
      "image_tokens": 0,
      "text_tokens": 0
    },
    "output_tokens": 0,
    "output_tokens_details": {
      "image_tokens": 0,
      "text_tokens": 0
    },
    "total_tokens": 0
  },
  "web_search": {
    "num_requests": 0
  }
},
"tools": [],
"top_logprobs": 0,
"top_p": 0.98,
"truncation": "disabled",
"usage": {
  "input_tokens": 959,
  "input_tokens_details": {
    "cache_write_tokens": 0,
    "cached_tokens": 0
  },
  "output_tokens": 494,
  "output_tokens_details": {
    "reasoning_tokens": 172
```

```
    },  
    "total_tokens": 1453  
  },  
  "user": null,  
  "metadata": {}  
}
```

状态码

状态码	含义	常见原因	处理建议
400	Bad Request	参数错误、JSON 格式错误	校验请求体
401	Unauthorized	API Key 错误或缺失	检查密钥和 Header
403	Forbidden	没有权限访问模型或资源	检查权限、账户状态
404	Not Found	路径或模型不存在	检查 Base URL、模型 ID
409	Conflict	资源状态冲突	稍后重试或检查任务状态
422	Unprocessable Entity	字段语义不合法	检查参数类型和范围
429	Too Many Requests	频率或额度限制	指数退避重试、限流
500	Internal Server Error	服务端异常	重试并记录 request id
503	Service Unavailable	服务暂不可用	稍后重试、切换模型

建议对 429 500 502 503 504 等自动重试

常见请求模板

```
{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "system",
      "content": "你是一个严谨、简洁、可靠的技术助手。回答必须基于用户提供的信息，不确定时说明不确定。"
    },
    {
      "role": "user",
      "content": "请总结下面的接口文档，并列出调用注意事项： ..."
    }
  ],
  "temperature": 0.2,
  "top_p": 1,
  "max_tokens": 1200,
  "stream": false
}
```

我们前面没有说到这个 **System Prompt**，其实也就是message的一部分。

因为不在这节课的重点，所以我们并没有论述它。

结束语

通过这一课，希望你已经十分清楚了，Agent底层其实就是一个简单的http调用。

至于做Agent到底还有多少门道，我们后面再细细道来

实现

Config实现

这个不需要说太多,

Message实现

message,就是我们每次发送给llm的那个message数组
在以往中,我们会把 **role**和 **content**加进去,
在这里我们需要额外封装三个其他

- **ToolCall**: 这个是LLM 给我们返回的工具调用决策,我们要加入到记忆中
- **ToolResult**: 这是我们放入记忆的 执行工具的结果
- **ThinkBlock**: 这是打开思考模式的推理信息,可以先不设计

在实现 mutil-tools的时候,发现tool_call总是null
查了很久,原因是: A畜的tool格式,对于openAI不适用

// 举个例子

```
{
  "model": "gpt-5.6-sol",
  "messages": [
    {
      "role": "user",
      "content": "读取 /home/code/README.md 的内容，然后帮我搜索一下其中提到的技术栈相关的最新信息"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "ReadFile",
        "description": "读取指定路径的文件内容，返回带行号的文件文本。适用于需要分析代码、日志或配置文件时。文件路径必须是绝对路径。",
        "parameters": {
          "type": "object",
          "properties": {
            "path": {
              "type": "string",
              "description": "文件的绝对路径，例如 /home/user/data.txt。不支持相对路径。"
            }
          }
        },
        "required": ["path"],
        "additionalProperties": false
      }
    },
    {
      "type": "function",
      "function": {
        "name": "WebSearch",
        "description": "搜索互联网信息，返回相关的网页摘要和链接。适
```

用于查询技术文档、最新资讯和问题解决方案。",

```
    "parameters": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "description": "搜索关键词"
        },
        "max_results": {
          "type": "integer",
          "description": "返回结果数量",
          "minimum": 1,
          "maximum": 10
        }
      },
      "required": ["query"],
      "additionalProperties": false
    }
  ],
  "tool_choice": "auto"
}
```

// 返回结果

```
{
  "id":
    "resp_0f660cd6efe582c1016a64aaaae7bc819aa6bc40e86552b24a",
  "object": "chat.completion",
  "created": 1784982186,
  "model": "gpt-5.6-sol",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "reasoning_content": "**Planning to read data first**",
        "tool_calls": [
          {
            "id": "call_SzRQRi58CjdJlHb1uRvUGCVN",
            "type": "function",
            "function": {
              "name": "ReadFile",
              "arguments": "{\n\"path\": \"\"/home/code/README.md\""}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls",
      "native_finish_reason": "tool_calls"
    }
  ],
  "usage": {
    "completion_tokens": 39,
    "total_tokens": 220,
    "prompt_tokens": 181,
    "prompt_tokens_details": {
      "cached_tokens": 0
    }
  }
}
```

```
    },  
    "completion_tokens_details": {  
      "reasoning_tokens": 14  
    }  
  }  
}
```

```
{  
  "role": "tool",  
  "tool_call_id": "call_SzRQRi58CjdJIHb1uRvUGCVN",  
  "content": "这里放 README.md 的实际文件内容"  
}
```

Coding教程不太好写,以后再写